Lab B.4 | IMU Sensor Fusion and Orientation Estimation
HI1033

Aim

The aim of this laboratory assignment is to investigate how different IMU-based sensor fusion methods affect orientation estimation in mobile system. Students will implement three orientation estimation algorithms and visualise pitch, roll and yaw in real time using internal and optionally external Bluetooth IMU sensors.

General

This assignment can be completed individually or in pairs.

To ensure quality of the implementation the solution should:

- Be structured using appropriate classes and architecture
- Follow MVVM or using similar architecture
- Separate sensor handling, orientation estimation and visualisation
- Contain clear documentation and code structure
- Submit source code before presentation

The Application

Students are asked to develop a mobile application capable of visualising orientation in real time using IMU sensors.

The application should:

- Read accelerometer, gyroscope and magnetometer data
- Implement three orientation estimation methods
- Visualise pitch, roll and yaw
- Compare multiple algorithms simultaneously
- Display graph-based orientation data

Orientation visualisation should be represented using moving objects displayed on the screen. The objects should respond to pitch, roll and yaw changes in real time. Different algorithms should be visualised simultaneously using separate objects to allow direct comparison between orientation estimation methods. The centre of the screen should represent the origin position of the visualisation.

Algorithms

Algorithm 1: EWMA filtered accelerometer-based orientation

For the first algorithm students should estimate pitch and roll using filtered accelerometer data only. An EWMA (Exponentially Weighted Moving Average) filter should be applied to reduce sensor noise and stabilise the orientation values.

The EWMA filter is defined as:

$$y_t = \lambda x_t + (1 - \lambda) y_{t-1} \qquad (1)$$

Where:

- y_t is the current filtered output
- y_{t-1} is the previous filtered output
- x_t is the current sensor input
- λ is the filter factor between 0 and 1

Students should observe how the algorithm behaves during both stationary and dynamic movement conditions. Since accelerometer data alone cannot estimate rotation around the z-axis, yaw should not be calculated in this algorithm.

Algorithm 2: Complementary filter

For the second algorithm, students should estimate pitch, roll and yaw by combining accelerometer and gyroscope data using a Complementary filter. The accelerometer contributes long-term stability while the gyroscope contributes smoother and faster rotational response.

The complementary filter is defined as:

$$\theta_t = \alpha(\theta_{t-1} + \omega_t \Delta t) + (1 - \alpha)\theta_{acc} \qquad (2)$$

Where:

- θ_t is the current orientation estimate
- θ_{t-1} is the previous orientation estimate
- ω_t is the angular velocity from the gyroscope
- Δt is the sampling interval
- θ_{acc} is the accelerometer-based orientation
- α is the filter factor between 0 and 1

Students should adjust the filter factor to reduce gyroscope drift while maintaining responsive orientation updates during movement.

Algorithm 3: Madgwick algorithm

For the third algorithm, students should implement Madgwick sensor fusion using accelerometer, gyroscope and magnetometer data. The algorithm estimates pitch, roll and yaw simultaneously in three dimensions using quaternion-based sensor fusion.

The Madgwick orientation update is defined as:

$$q_t = q_{t-1} + \dot{q}\Delta t - \beta\nabla f(q) \qquad (3)$$

Where:

- q_t is the updated quaternion orientation
- q_{t-1} is the previous quaternion orientation
- $\dot{q}$ is the quaternion derivative based on gyroscope data
- Δt is the sampling interval
- β is the correction factor
- $\nabla f(q)$ represents the gradient descent correction factor using accelerometer and magnetometer data

Students should observe how magnetometer data improves yaw estimation while also investigating how magnetic interference and calibration can affect orientation stability.

Basic requirements

To pass the application must include:

- Real-time visualisation of pitch, roll and yaw
- Display orientation values numerically
- Comparison between all three algorithms
- Graph-based visualisation
- Stable update of sensor values
- Start and stop the measurements

Higher level requirements

To receive a higher grade, the application should include at least two of the following:

- Ability to connect to a Bluetooth IMU sensor and switch between internal and external sensors
- Basic magnetometer calibration for improved yaw estimation

. Use Game Engine Godot for 3D visualization of orientation to create a simple orientation-matching game that visualizes pitch, roll and yaw.

Test Scenarios

Evaluate the algorithms during:

- Stationery position
- Slow movements
- Fast movements
- Rotation around the z-axis

Students should analyse:

- Stability
- Noise
- Drift
- Yaw estimation
- Motion responsiveness

Detailed Requirements

The following requirements are mandatory:

- The application should display continuously updated orientation values
- The user should be able to switch between the algorithms
- The user should be able to compare algorithms simultaneously
- Graphs should continuously in real time
- Orientation visualisation objects should respond continuously to pitch, roll and yaw changes

Resources

Godot Engine: https://godotengine.org/

Android implementation: https://github.com/Hadiseh04/Madgwick